

Introduction

This group project focuses on the research, understanding, evaluation and application of emerging game technologies within game development. As the industry continues to evolve rapidly, new tools, techniques and technological advancements are reshaping how games are developed.

This report will introduce what an emerging technology is, discuss how and why they should be implemented in game development, justify the chosen technology and how it incorporates into the groups game idea.

Lastly, the team's contributions of each member will be explained, the planning/ design of the teams created game and reflect on the outcome of the task, the challenges that may have occurred and any improvements if necessary.

Aim and Objectives

The aim of the project is to develop a small-scale interactive project, in this case a game, that shows understanding, implementation and use of an emerging game technology. Alongside the technical focus, the project also seeks to develop professional teamwork, communication and management in creating this game.

To achieve this, the project is structured around three objectives.

- Research and analysis: investigate a range of emerging game technologies, examining what they are, how they function, why they are significant within game development.
- Design and development: Plan, design and collaboratively build a game that showcases a selected technology.
- Evaluation and reflection: Assess the completed project, review the effectiveness of the team's collaboration, and reflect on the challenges encountered and the improvements that could be made in future work.

Concept and Background

Procedural Content Generation (PCG) has emerged as a significant technique within the game development industry, enabling the automated creation of in-game content through algorithmic processes. As the scale and complexity of modern games continues to evolve, PCG provides developers with a means of generating expansive, varied and replayable environments while reducing the reliance on manual asset creation.

The project adopts PCG as its core technological focus, implemented within a first-person shoot game developed within the Unity engine. The game places the player within a generated open-world environment, where they must survive successive waves of enemies (zombies). Core gameplay elements including terrain and environmental obstacles spawning, enemy spawn locations and resource pickups like health and ammo are all generated procedurally to ensure each playthrough offers a unique experience. This design directly aligns with the objective of PCG in being to enhance replay ability and variability through controlled pseudo-randomness.

PCG is the use of algorithms and techniques that automatically generates content in games such as terrain, maps, levels and textures, even game elements such as enemies, events and narratives (MONTSERRAT, 2025). Due to its automatic generation that can create and spawn on a mass level, it allows developers to create say, a randomly generated universe for players to have a unique experience in *No Mans Sky*, then there is *Minecraft* which uses PCG to generate vast, almost endless open worlds for players to explore, but also provide that unique experience (MONTSERRAT, 2025).

However, the implementation of PCG also introduces some challenges. A primary limitation lies in the potential lack of design control, as procedurally generated content may not meet the intended quality or gameplay standards. Without careful constraint systems in place, PCG can result in repetitive or incoherent environments, diminishing player experience. Furthermore, the balance between randomness and structure is vital, excessive randomness can undermine gameplay clarity while overly constrained systems may negate the benefits of procedural generation altogether.

In comparison, to machine learning-based content generation, PCG offers a more controllable approach. While machine learning techniques such as neural networks and generative adversarial networks can produce highly complex and adaptive content, they require substantial datasets and computer resources. For small scale projects, such as the one presented in this report, PCG provides a more practical and efficient solution, allowing for meaningful procedural systems without the overhead associated with machine learning models.

PCG plays a big role in game development through so many means, this is why the project will focus on it, to represent how it works, why it is good and what it can offer. PCG is implemented into the projects game but utilising it to generate the games terrain that the level will be played up and each time the terrain will be different in terms of layout on each new playthrough. Next, it will also automatically generate the spawn of the enemies and spawn them randomly through the level, alongside any obstacles the player may face such as trees, rocks or cacti. The terrain itself is an open map grassland that will features ponds/ lakes, hills. Depending on the height of the terrain itself it will spawn specific obstacles for that area as the game will feature different biome designs, such as spawning the cacti in the desert biome, snowing being painted on the terrain on the highest peaks of the terrain. The game features player picks up items such as health and ammo packs that spawn at random throughout the level.

Roles and Responsibilities

When it came to the team's roles and responsibilities, each member chose on what there were strengths are and how they could implement and provide great teamwork for the project. All roles and responsibilities were agreed upon by everyone in the group, even though there were specific roles, the team worked closely together and was able to work as a team throughout most of the project, which allowed for the game to development and represent the project's purpose successfully. There was no specific team leader as the team itself again, worked very closely together and always kept in contact to keep each other up to date with any developments made or issues faced.

Lukas Kubinec was to create the PCG that would be used to generate the terrain for the level and generating the spawn points across it too, using noise maps. Also using PCG to generate the obstacles of the level and any vegetation and their spawning by using rule-based cell collapse (simplified Wave Function Collapse). Finally, the creation of the games GUI (Graphical User Interface) which will represent all the players stats such as health, ammo, round number etc.

Luke Broadley was to create the games dynamic difficulty adjustment (DDA), this is a technique that alters the challenge of the game for the player by modifying the games AI behaviour and mechanics based on the players performance of the game. Another responsibility was to create the games back-end logic that takes care of most of the game's calculations and their implementation into the game, keeping the game functioning and flowing smoothly. Luke was also in charge of the 3D modelling and asset creation for the game such as trees, cactus, rocks, enemy design etc which were created in Blender. Finally, to add some extra effect to the enemy design of the game, Luke performed some voice lines for the zombies, 4 different types of zombie noises to add to the enemies.

Dylan Dwyer was to work on the health and ammo pickups that spawn randomly throughout the level. Then to create the health, lives and ammo management system, the GUI system displays the players health lives and ammo. Finally in charge of managing the groups project report.

Tools and Technologies in Development

World Generator System

This is the PCG known as the world generator system, which uses PCG algorithms, specifically Perlin, Cnoise, Snoise noise generators to procedurally generate terrain, terrain heightmaps, biome texturing, and the placement of enemies and obstacles. The Start() method initialises references to the GameManager, the Unity Terrain obstacles, and the NavMeshSurface, ensuring the generator has access to all the required systems it needs to perform.

The script provides several methods such as GetTerrainObject() exposes the generated terrain to the other systems, GetWorldIsReady() allows the game manager to synchronise gameplay with the world generation, and BakeNavigation() rebuilds the NavMesh after the terrain has been modified.



```
67
68     private void Start()
69     {
70         gameManager = GameManager.instance;
71         navMesh = GetComponent<NavMeshSurface>();
72
73         WorldTerrain = GetComponent<Terrain>(); // Assings the required components
74     }
75
76     public Terrain GetTerrainObject()
77     {
78         return WorldTerrain;
79     }
80
81     public bool GetWorldIsReady()
82     {
83         return worldIsReady;
84     }
85
86     public void BakeNavigation()
87     {
88         navMesh.BuildNavMesh();
89     }
90
```

Figure 1 World Generator Script Screenshot

The Procedural vegetation is then handled by the CreateGrass(), which samples the terrain heightmap and then populates a detail layer with gras only in regions that fall between the sand and grass biome areas.

```

public void CreateGrass()
{
    int detailRes = WorldTerrain.terrainData.detailResolution;
    float terrainMaxHeight = WorldTerrain.terrainData.size.y;

    //float[,] heightmapData = WorldTerrain.terrainData.GetHeights(0, 0, detailRes, detailRes);
    int[,] generatedGrassMap = new int[detailRes, detailRes];

    for (int y=0; y < detailRes; y++)
    {
        for (int x = 0; x < detailRes; x++)
        {
            float normX = x / (float)detailRes;
            float normY = y / (float)detailRes;

            // Get heigh at point
            float worldHeight = WorldTerrain.terrainData.GetInterpolatedHeight(normX, normY) / terrainMaxHeight;

            if (sandHeightLevel < worldHeight && worldHeight < grassHeightLevel)
            {
                generatedGrassMap[y,x] = 25;
            } else
            {
                generatedGrassMap[y, x] = 0;
            }
        }
    }

    WorldTerrain.terrainData.SetDetailLayer(0,0,0, generatedGrassMap);
}

```

Figure 2 World Generator Script CreateGrass()

The main generation pipeline is handled in `BeginGenerateTerrainAndSpawns()`, which applies optional random seed offsets, prepares texture sizes, generates noise based heightmaps, constructs the terrain surface and computes the spawn locations for the zombies and obstacles.

The spawn point generation is performed by `BeginGenerateSpawnPoints()`, which scans a noise map and selects positions below a threshold value, applying an additional probability check before sampling the terrain height and recording the final spawn coordinates. Other methods such as `CheckRandomSeed()` and `PrepareTexture()` configure noise offsets and texture dimensions, while `GenerateTerrain()` applies the heightmap and paints the biomes correctly using the `SelectColour()` function. Finally, `GenerateNoiseMap()` produces the noise based heightmaps using Perlin, classic Perlin and Simplex noise, normalising and clamping values to ensure consistency in terrain generation.

Terrain Object Spawn Manager

The `TerrainObjectSpawnManager` is the script responsible for populating the procedurally generated terrain with the environmental obstacles and the gameplay pickups. It operates alongside the `WorldGenerator` script by using the terrain's biome height thresholds and pre-computed spawn locations to ensure the objects are placed correctly and logically without any overlap issue.

As noted before the obstacle spawning is controlled through a height-based biome system, so rocks appear in water regions, rocks and cacti in sand regions and trees in grass and mountain regions. Spawn points are selected from the WorldGenerators obstacle spawn map, each object is aligned to the terrain surface using ray casting to ensure natural placement.

Pickup spawning (health and ammo) uses a randomised placement system within the terrain boundaries. Each location is validated using collision checks to prevent any overlapping with existing objects. Pickups continue to spawn throughout the gameplay until the required number which is defined within the skilled manager is reached.

Procedural Obstacle Generator

The ProceduralObstacleGenerator script implements a grid based PCG system for constructing obstacles such as rocks, cacti, or tree. The generator operates on a three-dimensional grid, where each cell can collapse into one of the several states (empty or filled) based on the rules and probability values. During initialisation, the script repeatedly selects random un collapsed cells and applies rule based constraints to determine whether the cell becomes part of the structure. These rules include support requirements (ensuring upper cells are placed only when lower cell is filled), enforced core structures (used for cactus and trees), and directional side branch placement.

Once all the cells have collapsed, the BuildModel() method implements the appropriate mesh parts into the world. Mesh selection is determined by the cells position and state, while rotation rules allow for randomised orientation or controlled alignment towards the centre.


```

void BuildModel()
{
    //var ObstaclesParent = new GameObject("ObstaclesParent");
    var pos = this.gameObject.transform.position;

    for (int z = 0; z < sizeZ; z++)
    {
        for (int y = 0; y < sizeY; y++)
        {
            for (int x = 0; x < sizeX; x++)
            {
                // Rocks
                if (cellMustBeSupported)
                {
                    if (genGrid[x, y, z] > 1)
                    {
                        // Piles rocks from largest to smallest - Must be set in Inspector in this order!
                        InstantiatePrefab(x, y, z, pos, meshParts[y], 0.0f);
                    }
                }

                // Builds the core parts of Cactus or Trees (0 - bottom core / 1 - top core / 2 - side)
                if (centreMustBeOneType)
                {
                    if (x == 1 && z == 1)
                    {
                        // Builds the bottom core part
                        if (y == 0)
                        {
                            InstantiatePrefab(x, y, z, pos, meshParts[0], 0.0f);
                        } else if (y == 1)
                        {
                            // Checks if there is a fill cell above
                            if (genGrid[x, y+1, z] > 1)
                            {
                                // If there is a fill cell above, this cell gets bottom core part
                                InstantiatePrefab(x, y, z, pos, meshParts[0], 0.0f);
                            } else
                            {
                                // If cell above is empty, this cell gets the top core part
                                InstantiatePrefab(x, y, z, pos, meshParts[1], 0.0f);
                            }
                        } else if (y == 2 && genGrid[x, y, z] > 1)
                        {
                            // Gets the top core part
                            InstantiatePrefab(x, y, z, pos, meshParts[1], 0.0f);
                        }
                    }
                }

                // Builds sides/branches around core
                if (sideCellsMustBeRotatedTowardCentre && genGrid[x, y, z] > 1)
                {
                    if (x == 0 && z == 0)
                    {
                        InstantiatePrefab(x, y, z, pos, meshParts[2], 135.0f);
                    } else if (x == 1 && z == 0)
                    {
                        InstantiatePrefab(x, y, z, pos, meshParts[2], 90.0f);
                    } else if (x == 2 && z == 0)
                    {
                        InstantiatePrefab(x, y, z, pos, meshParts[2], 45.0f);
                    } else if (x == 0 && z == 1)
                    {
                        InstantiatePrefab(x, y, z, pos, meshParts[2], 180.0f);
                    } else if (x == 2 && z == 1)
                    {
                        InstantiatePrefab(x, y, z, pos, meshParts[2], 0.0f);
                    } else if (x == 0 && z == 2)
                    {
                        InstantiatePrefab(x, y, z, pos, meshParts[2], 225.0f);
                    } else if (x == 1 && z == 2)
                    {
                        InstantiatePrefab(x, y, z, pos, meshParts[2], 270.0f);
                    } else if (x == 2 && z == 2)
                    {
                        InstantiatePrefab(x, y, z, pos, meshParts[2], 315.0f);
                    }
                }
            }
        }
    }
}

```

Figure 3 BuildModel() method.

Obstacle Generator

The ObstacleGenerator script implements a simplified three-dimensional version of the Wave Function Collapse (WFC) algorithm to procedurally assemble obstacles. The generator operates on a fixed 3x3x3 grid, where each cell initially contains a dull set of possible tile states. During the execution, the system repeatedly selects the cell with the lower entropy, collapses it to a single tile, and then propagates adjacency constraints to the neighbouring cells.

Once all the cells have collapsed, the BuildModel() method instantiates mesh components into the world based on the final configuration. Each filled cell spawns a mesh part at its corresponding grid coordinate, with a small random rotation applied to introduce visual variation. This enables the creation of structured obstacles using a compact rule set and a modular asset library.

```
void BuildModel()
{
    var pos = this.gameObject.transform.position;

    for (int z = 0; z < size; z++)
    {
        for (int y = 0; y < size; y++)
        {
            for (int x = 0; x < size; x++)
            {
                if (grid[z,y,x][0] != 0)
                {
                    float randRot = Random.Range(-1, 1);
                    var newRot = new Quaternion(randRot, randRot, randRot, 1);
                    var newObs = Instantiate(meshParts[y], pos, newRot);

                    newObs.transform.SetParent(this.transform);
                    var goPos = new Vector3(x, y, z);
                    newObs.transform.localPosition = goPos;
                }
            }
        }
    }
}
```

Figure 4 BuildModel() Script in ObstacleGenerator.

Graphical User Interface



Figure 5 in game GUI

The game's GUI is responsible for updating and showcasing all the players/ game's stats during the gameplay. It is managed and controlled by the UIManager script. It manages the UI panels displaying the stats. Each UI element is exposed through TextMeshProUGUI fields, allowing the script to modify the text dynamically in response to gameplay events.

The class provides dedicated updates for each category of information being displayed. These methods format numerical values into readable strings and assign them to the correct UI component. The script also includes controls for enabling or disabling the loading and game-over screens, as well as a restart function to reloads the current scene and resets the game.

Game Manager

The GameManager script functions as the central coordinator for all the gameplay systems implemented, ensuring that the world generation, navigation baking, player spawning and enemy spawning occur in a controlled and non-conflict sequence. It maintains references to all major subsystems created including terrain generation, obstacle spawning, skill and health management, UI control, and character input.

The core logic of the script is implemented in `GameStartInOrder()`, which executes a staged initialisation pipeline. First, the world generator constructs the procedural terrain and spawn maps. Once terrain generation is complete, the obstacle manager populates the environment, after which the navigation mesh is baked to allow AI to navigate the newly created world. The player is then spawned at the centre of the map, the loading screen is removed, and the zombies will start spawning to the number required for the round.

```
private void GameStartInOrder()
{
    // The first initial script that starts all the necessary components in one order, in order to minimise the conflicts.
    if (!worldGenerated)
    {
        cameraEffects.SetLoadingScreen(true);
        worldGenerator.BeginGenerateTerrainAndSpawns();
        if (worldGenerator.GetWorldIsReady()) { worldGenerated = true; } // Continues only if the world is built
    } else if (worldGenerated && !objectSpawnManager.AllObstaclesSpawned())
    {
        worldGenerator.CreateGrass(); // Creates grass meshes on terrain
        objectSpawnManager.SetEnableObjectSpawn(true);
    }
    else if (objectSpawnManager.AllObstaclesSpawned() && !NavigationBaked)
    {
        worldGenerator.BakeNavigation();
        NavigationBaked = true;
    }
    else if (NavigationBaked && !PlayerIsSpawned)
    {
        skillManager.RespawnPlayerCentred();
        PlayerIsSpawned = true;
        cameraEffects.SetLoadingScreen(false);
    }
    else if (PlayerIsSpawned)
    {
        zombieSpawnManager.SetZombieSpawn(true);
        uiManager.ChangeStateOfLoadingUI(false); // Hides the Loading UI
        gameIsReady = true; // World initiation is complete
    }
}
```

Figure 6 `GameStartInOrder()` method in `GameManager`.

Skill Manager (Dynamic Difficulty Adjustment System)

The SkillManager script manages the game's dynamic difficulty adjustment system by adjusting the zombie attributes, wave size, spawn rates, and player related stats in response to performance. It tracks gameplay metrics including kills, accuracy, and wave progression and uses these values to compute the number of enemies in the next wave and the rate at which they spawn. It also recalculates the zombie's health, speed and damage each wave, ensuring that enemy difficulty scales correctly with the player's success while remaining capped by predefined values.

The script also manages accuracy tracking by recording successful and total hits, updating the UI accordingly. It also provides a centralised method for respawning the player at the terrain's midpoint, sampling the terrain height to ensure correct placement. The SkillManager also communicates with UIManager and the ZombieSpawnManager to update the interface elements and reset wave related counters.

Zombie Spawn Manager

The ZombieSpawnManager and the ZombieData collectively manage enemy instantiation, navigation and wave based spawning behaviour. ZombieData functions as a data container that retrieves the zombie's stats, such as health, speed and damage from the SkillManager, ensuring that each spawned zombie inherits the current difficulty starts.

The ZombieSpawnManager oversees the timed spawning of zombies during each wave. It maintains the counters for the number of zombies spawned, enforces a spawn rate, and prevents zombies from appearing too close to the player by using collision checks around the candidate spawn points. The spawn locations are sourced from the procedural terrain generator, ensuring spatial variety across the playthroughs. Each zombie is instantiated under a dedicated parent object for organisational clarity, assigned a ZombieData instance, and immediately directed towards the player using Unity's navigation system. The manager also detects when the full wave has been spawned and continuously updates each zombie's destination, enabling persistent pursuit.

Health Manager

The health manager script is responsible for tracking and updating the players health, lives and death related state changes. It maintains the players current and maximum health values, processes incoming damage, and applies healing effects from health pick-ups. When the player picks up the corresponding health or ammo pick up, the CharacterTriggerCollisionController script removes the object from the scene, updates the player resource through the WeaponManager or HealthManager, and notifies the TerrainObjectSpawnManager so that new pickups can be spawned. Whenever health changes, the script updates the corresponding UI element within the UIManager, ensuring the change of data is visualised to the player.

Reflections

This project provided valuable insight into both the technical and collaborative challenges associated with the development of this project. Overall, the team successfully delivered a functional prototype that demonstrates the practical application of PCG systems in a game, particularly in terrain generation, environmental population, and dynamic gameplay elements.

From a technical perspective, the integration of multiple PCG systems such as the terrain generation, obstacle generation and placement, and enemy spawning was a key achievement. The use of noise-based algorithms combined with rule-based constraints enabled the creation of varied and coherent environments across each playthrough. This demonstrated a clear understanding of how PCG can be used not only to automate content creation but also to enhance replay ability and showcase the advantage it has on development.

From a teamworking perspective, the group demonstrated effective communication and collaboration throughout the development process. Regular updates and shared access to the project files ensured all members remained aligned and aware of ongoing progress. The early agreement on the project concept and scope was particularly beneficial too, as it reduced the need for significant redesign during development.

In terms of the project scope, while the implemented systems are functional and work as intended for the purpose of the project, there is some potential for further expansion. The addition of more diverse enemy types, more complex environmental interactions or procedurally generated structures could significantly enhance the gameplay depth.

For example, the terrain generation could also be enhanced, as currently the terrain is smooth, with no steep hills nor deep valleys.

In conclusion, the project successfully demonstrates the practical application of procedural content generation within a game development setting, achieving its objectives.

References

MONTSERRAT, R., 2025. *Procedural Content Generation for video games, a friendly approach* [online]. Available from: <https://www.levelup-gamedevhub.com/en/news/procedural-content-generation-for-video-games-a-friendly-approach/> [Accessed 30 April 2026].

IXIE, 2026. *PCG + Telemetry: The Feedback Loop That Makes Infinite Content Actually Work* [online] Available from: <https://www.ixiegaming.com/blog/pcg-telemetry-the-feedback-loop-that-make-infinite-content-actually-work/> [Accessed 30 April 2026].